



湖南理工学院
Hunan Institute of Science and Technology

单例模式

课 程： 软件体系结构

班 级： 软件 21-3BF

姓 名： 张道

学 号： 12214801130

日 期： 2024-5-11

1、单例模式简介

单例模式是一种创建型设计模式，用于确保一个类只有一个实例，并提供一种访问该实例的全局方式。它通常适用于需要共享资源的情况，例如数据库连接、线程池、配置对象、文件系统、资源管理等。

2、单例模式问题

单例模式解决了以下两个问题：

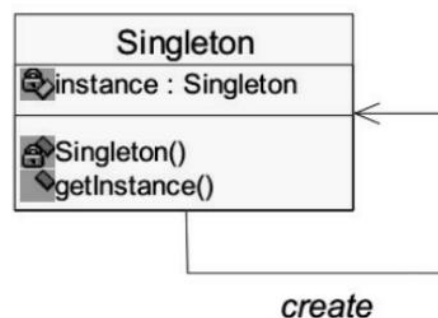
- 保证一个类只有一个实例。这通常用于控制对共享资源的访问，以确保资源的一致性和可靠性。
- 提供一个全局访问节点。通过单例模式，客户端可以通过统一的接口获取到该类的唯一实例，而无需关心实例是如何创建和管理的。

3、单例模式实现方法

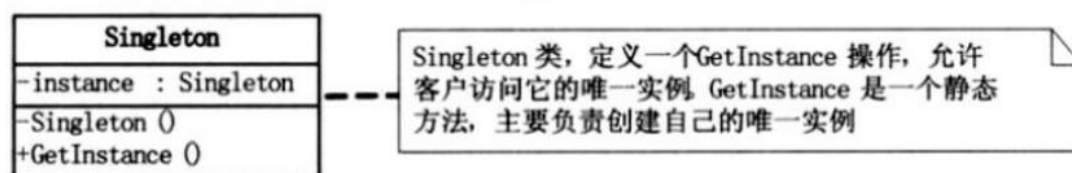
□ 单例模式的实现方法

- 构造函数变为私有，限制外部通过 **new** 来实例化对象
- 提供一个自己的对象以及访问这个对象的静态方法
- **懒汉式**和**饿汉式**
- 客户通过调用类方法来得到类的对象

类图结构



类图解析：



4、单例模式解决方案

单例模式的实现通常包括以下几个步骤：

- 将类的构造方法私有化，以防止外部代码直接实例化该类。
- 提供一个静态方法来获取类的唯一实例，在这个方法中进行实例的创建和返回。
- 在静态方法中对实例进行延迟加载，即在第一次调用时才创建实例，以节省资源并提高效率。
- 考虑多线程环境下的安全问题，可以通过加锁或使用双重检查锁等方式来确保在并发情况下也能正确地获取单例实例。

5、单例模式的应用场景

单例模式适用于以下场景：

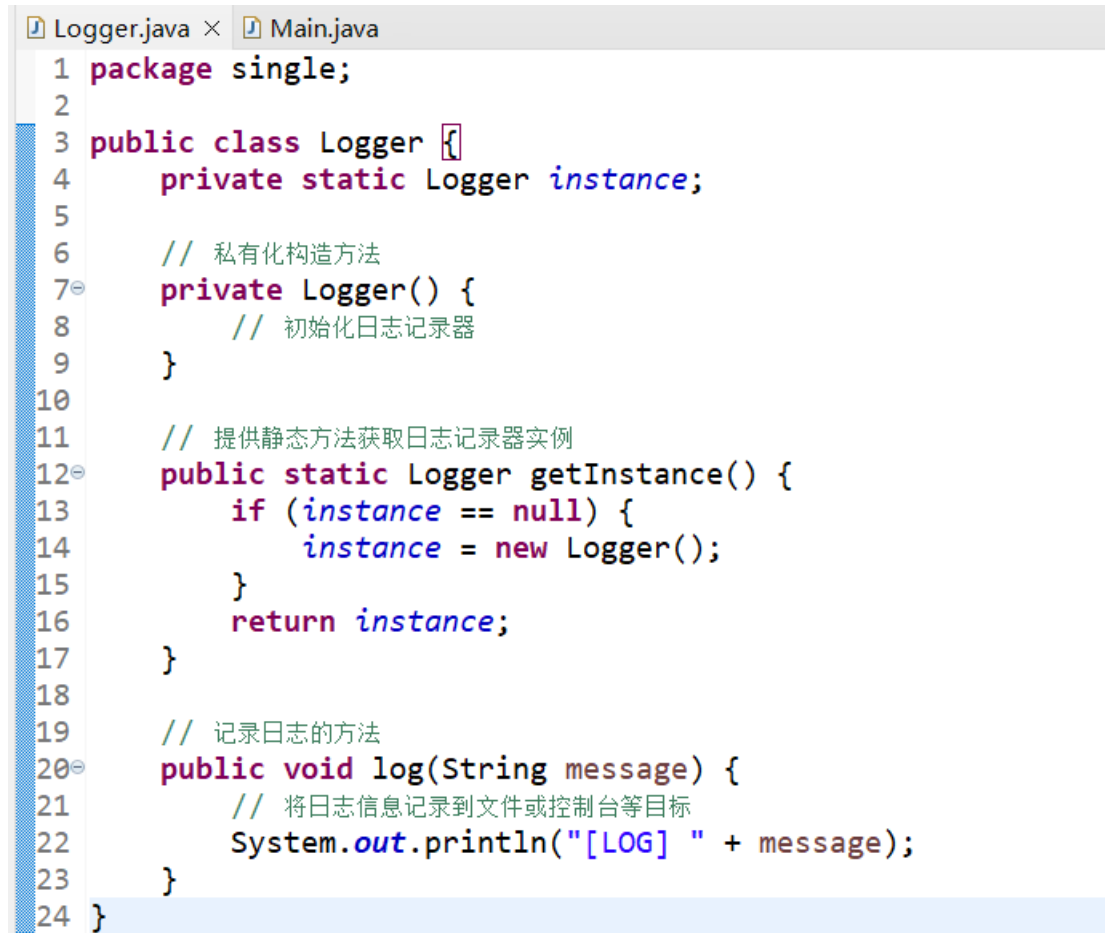
- 当一个类只有一个实例，并且需要提供一个全局的访问节点时。
- 当需要控制全局资源的访问权限时，可以使用单例模式来确保资源的一致性和可靠性。

6、现实世界的类比

政府是单例模式的一个典型示例。每个国家只有一个官方政府，而政府代表着该国的权力和管理机构，因此可以被视为一个单例实例。

7、示例代码

以下是一个典型的单例模式示例代码：



```
1 package single;
2
3 public class Logger {
4     private static Logger instance;
5
6     // 私有化构造方法
7     private Logger() {
8         // 初始化日志记录器
9     }
10
11     // 提供静态方法获取日志记录器实例
12     public static Logger getInstance() {
13         if (instance == null) {
14             instance = new Logger();
15         }
16         return instance;
17     }
18
19     // 记录日志的方法
20     public void log(String message) {
21         // 将日志信息记录到文件或控制台等目标
22         System.out.println("[LOG] " + message);
23     }
24 }
```

在这个示例中：

Logger 类是一个单例类，它只有一个私有的静态成员变量 instance 用于保存单例实例。

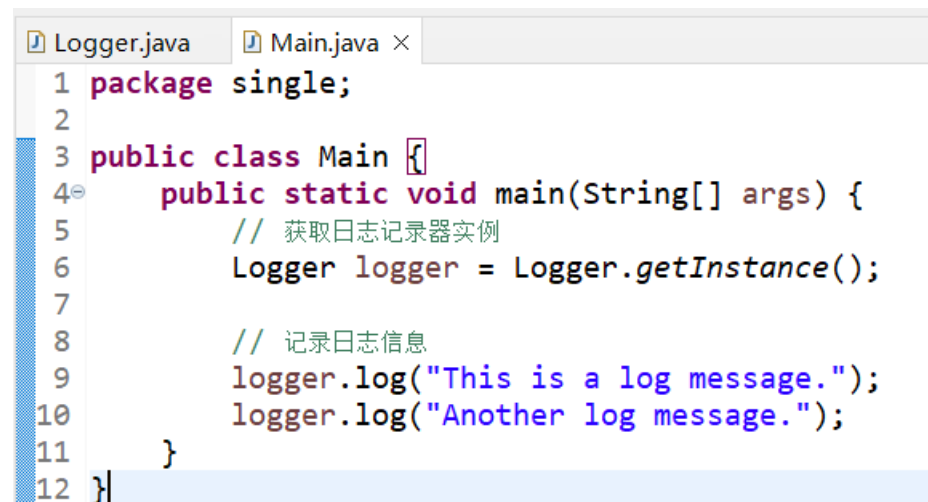
构造方法 Logger() 被私有化，这意味着外部代码无法直接通过 new 关键字实例化 Logger 对象，从而确保了该类只有一个实例。

静态方法 getInstance() 被提供用于获取 Logger 类的唯一实例。在该方法中，通过检查 instance 是否为 null，如果是，则创建一个新的 Logger 实例并将其赋值给 instance，然后返回这个实例；如果不是，则直接返回已有的

实例。

`log(String message)` 方法用于记录日志信息，将日志信息输出到控制台。

客户端可以通过调用该方法来记录日志信息。



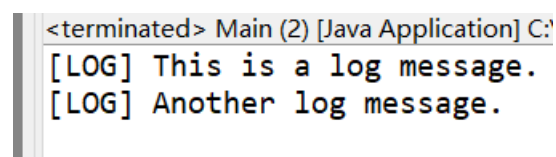
```
1 package single;
2
3 public class Main {
4     public static void main(String[] args) {
5         // 获取日志记录器实例
6         Logger logger = Logger.getInstance();
7
8         // 记录日志信息
9         logger.log("This is a log message.");
10        logger.log("Another log message.");
11    }
12 }
```

Main 类是客户端代码的入口，其中的 main 方法用于演示如何使用 Logger 类的单例实例。

首先调用 `Logger.getInstance()` 方法获取 Logger 类的唯一实例，并将其赋值给变量 `logger`。

然后调用 `logger.log(String message)` 方法记录日志信息，这里演示了两次日志记录操作。

运行结果：



```
<terminated> Main (2) [Java Application] C:
[LOG] This is a log message.
[LOG] Another log message.
```

通过这两段代码，我们可以看到单例模式的使用方式。客户端代码通过调用单例类的静态方法来获取唯一的实例，并通过该实例来访问类的其他方法和属

性，从而实现了全局唯一对象的访问和操作。

8、总结

单例模式是一种简单但非常有用的设计模式，它可以帮助我们管理全局唯一的对象或资源，并确保它们在整个系统中的一致性和可靠性。通过限制类的实例化过程，单例模式确保在整个系统中只有一个实例存在，并提供一个全局的访问节点来获取这个实例。